Structured Streaming Programming Guide (../streaming/index.html) «

- Overview (../streaming/index.html)
- Getting Started (../streaming/getting-started.html)
- APIs on DataFrames and Datasets (../streaming/apis-on-dataframes-and-datasets.html)
- Performance Tips (../streaming/performance-tips.html)
- Additional Information (../streaming/additional-information.html)

Structured Streaming Programming Guide

Overview

Structured Streaming is a scalable and fault-tolerant stream processing engine built on the Spark SQL engine. You can express your streaming computation the same way you would express a batch computation on static data. The Spark SQL engine will take care of running it incrementally and continuously and updating the final result as streaming data continues to arrive. You can use the Dataset/DataFrame API (../sql-programming-guide.html) in Scala, Java, Python or R to express streaming aggregations, event-time windows, stream-to-batch joins, etc. The computation is executed on the same optimized Spark SQL engine. Finally, the system ensures end-to-end exactly-once fault-tolerance guarantees through checkpointing and Write-Ahead Logs. In short, *Structured Streaming provides fast, scalable, fault-tolerant, end-to-end exactly-once stream processing without the user having to reason about streaming.*

Internally, by default, Structured Streaming queries are processed using a *micro-batch processing* engine, which processes data streams as a series of small batch jobs thereby achieving end-to-end latencies as low as 100 milliseconds and exactly-once fault-tolerance guarantees. However, since Spark 2.3, we have introduced a new low-latency processing mode called **Continuous Processing**, which can achieve end-to-end latencies as low as 1 millisecond with at-least-once guarantees. Without changing the Dataset/DataFrame operations in your queries, you will be able to choose the mode based on your application requirements.

In this guide, we are going to walk you through the programming model and the APIs. We are going to explain the concepts mostly using the default micro-batch processing model, and then later (./performance-tips.html#continuous-processing) discuss Continuous Processing model. First, let's start with a simple example of a Structured Streaming query - a streaming word count.

Structured Streaming Programming Guide (../streaming/index.html) «